FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of 14 JUNE 2021 - Part Theory

LAST + FIRST NAME:

(capital letters please)

MATRICOLA: SIGNATURE:

(or PERSON CODE)

- The exam is in ***written form*** and consists of ***two parts***:
- ***Theory*** (80%): Syntax and Semantics of Languages

    1. regular expressions and finite automata
    2. free grammars and pushdown automata
    3. syntax analysis and parsing methodologies
    4. language translation and semantic analysis

- ***Lab*** (20%): Compiler Design by Flex and Bison
- To pass the ***exam***, the candidate must succeed in ***both*** parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass the ***theory*** part, the candidate must sufficiently answer all ***four*** parts.

PART 1 - regular expressions and finite automata

1. Consider the ***regular expression*** **R** below:

$$R = a\ (\ b \mid cc\ )^*\ b$$

Find the correct ***initials***, ***finals*** and ***digrams*** of **R**.

Fin (R) =

Dig (R) =

Ini (R) =

2. Is the ***regular language*** **L(R)** of the regexp **R** below:

$$R = a\ (\ b \mid cc\ )^*\ b$$

local ?

Select one:

- True
- False

3. Explain in short your answer to the previous question, whether the language of the regexp **R** below:

$$R = a\ (\ b \mid cc\ )^*\ b$$

is local or not.

4. Is the regexp **R** below:

$$R = a\ (\ b \mid cc\ )^*\ b$$

ambiguous ?

Select one:

- True
- False

5. Explain in short your answer to the previous question, whether the regexp ***R*** below:

$$R = a\ (\ b \mid cc\ )^*\ b$$

is ambiguous or not.

6. Consider ***another regular expression R'*** below:

$$R' = a\ (\ b \mid cc \mid \varepsilon\ )^+\ b$$

and select the ***correct*** statements below.

Select one or more:

- ☐ language **L(R')** is ***not local***
- ☐ regexp **R'** is ***ambiguous***
- ☐ regexp **R'** is ***not equivalent*** to regexp **R**
- ☐ regexp **R'** is ***equivalent*** to regexp **R**
- ☐ regexp **R'** is ***not ambiguous***

7.

Consider now the ***numbered version*** $R_{\#}$ of the above regular expression ***R***, terminated with the end-mark symbol ***t***. Notice that usually the end-mark symbol ***t*** is denoted as ⊣.

$$R_{\#} = a_1 \, (\, b_2 \mid c_3 c_4 \,)^* \, b_5 \, t$$

Choose the ***sets*** necessary for the construction of the ***Berry-Sethi recognizer***.

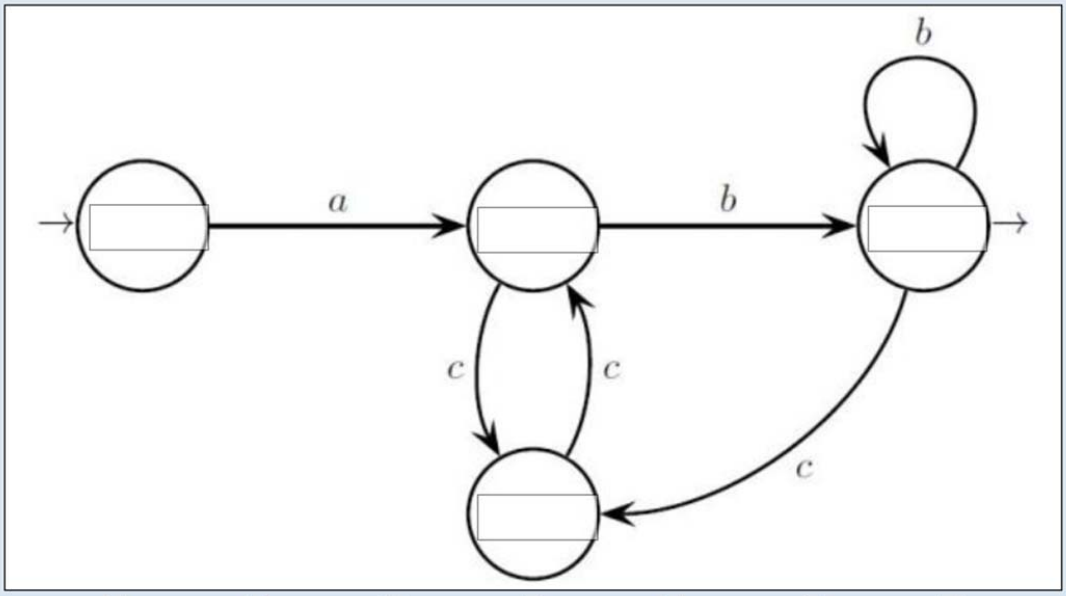
Ini ($R_{\#}$)	Fol (a_1)	Fol (b_2)	Fol (c_3)	Fol (c_4)	Fol (b_5)
Choose...	Choose...	Choose...	Choose...	Choose...	Choose...
b2, c3, b5	b2, c3, b5	b2, c3, b5	b2, c3, b5	b2, c3, b5	b2, c3, b5
b5	b5	b5	b5	b5	b5
c4	c4	c4	c4	c4	c4
b2, c3	b2, c3	b2, c3	b2, c3	b2, c3	b2, c3
t	t	t	t	t	t
a1	a1	a1	a1	a1	a1

8.

For the above regexp **R**:

$$R = a \, (\, b \mid cc \,)^* \, b$$

place the correct initials and followers into the ***Berry-Sethy automaton graph*** below:

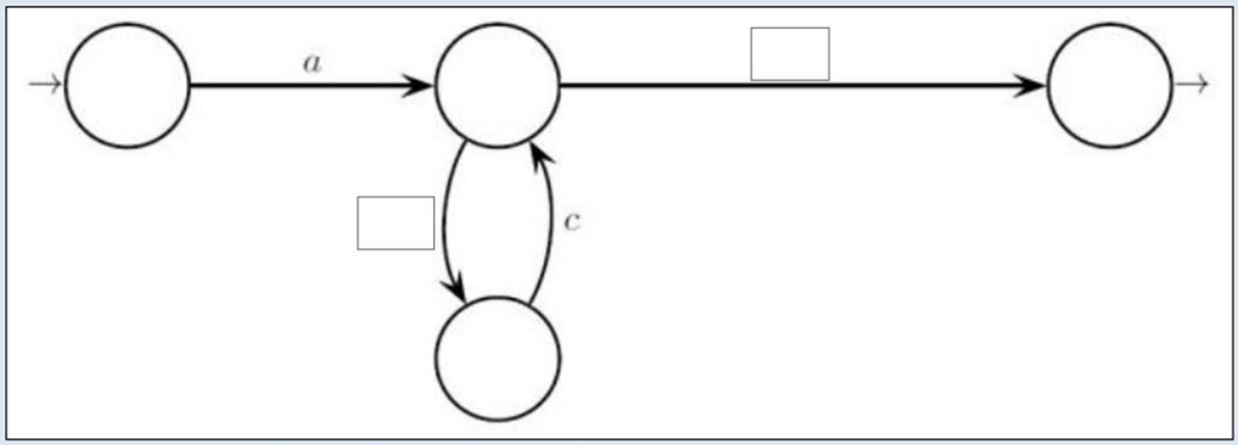


a1 | b2 c3 b5 | b2 c3 b5 t | c4 | b2 c3 | b5 t | c3 c4 | b2 b5

b2 c4 t | t

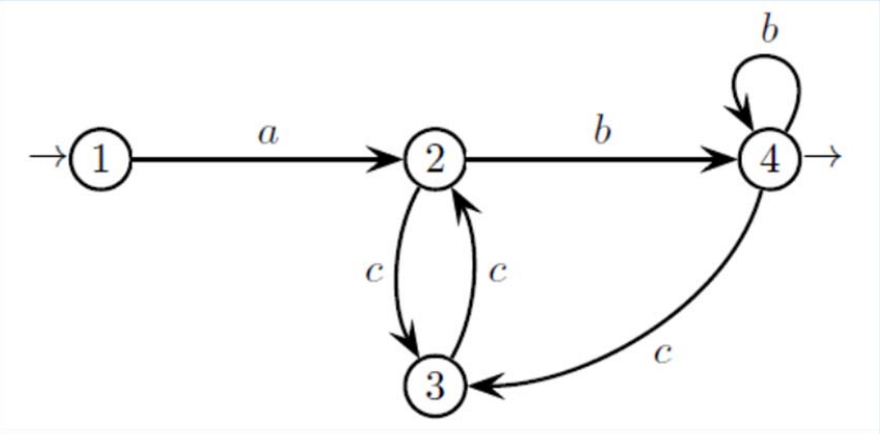
9.

Consider the automaton below, with generalized transitions (i.e., transitions are labeled with regular expressions). Choose the regular expressions to place on the transitions so that the resulting automaton is equivalent to the Berry Sethi automaton of the previous question.



b* | b+ | b+c | b*c | ccb* | ccb+ | bc

10. Consider the following ***automaton***, equivalent to the regexp **R** of the previous questions:



Write an ***equivalent*** system of linear language ***equations***.

11. Now ***solve*** the equation system written in the previous question and find a ***regexp*** for the language ***L(1)***.

PART 2 - Free grammars and pushdown automata

Consider the language L_1 over the alphabet **{a, b}** consisting of the strings of positive even length such that if at any position there is an ***a*** symbol (respectively, a ***b*** symbol), then at the symmetric position with respect to the string center there is a ***b*** symbol (respectively, an ***a*** symbol).

For instance, $abbaab \in L_1$, while $\varepsilon,\ aa,\ abaa,\ aab \notin L_1$.

12. Write a ***BNF grammar***, non-extended, for language L_1.

13. Say which is the ***simplest*** kind of ***automaton*** that can recognize language L_1.

Select one:

- deterministic push-down automaton
- none of the other options
- nondeterministic push-down automaton
- finite-state automaton

14. Say which is the ***smallest family*** of languages that includes language L_1.

Select one:

- deterministic context-free languages
- regular languages
- none of the above
- context-free languages
- general, unrestricted languages

Now consider a language L_2 over the alphabet **{a, b, c}** consisting of the strings of positive odd length such that the central letter is a ***c*** and every other letter is an ***a*** or a ***b***, and if at any position there is an ***a*** symbol (respectively, a ***b*** symbol) then at the symmetric position with respect to the central *c* there is a ***b*** symbol (respectively, an ***a*** symbol).

For instace, $c,\ babcaba \in L_2$, while $\varepsilon,\ abcbb,\ abacb \notin L_2$.

15. Write a ***BNF non-extended grammar*** for language L_2.

16. Say which is the ***simplest type*** of ***automaton*** that can recognize language L_2.

Select one:

- none of the other options
- deterministic push-down automaton
- nondeterministic push-down automaton
- finte-state automaton

17. Say which is the ***smallest family*** of languages that includes language L_2.

Select one:

- deterministic context-free languages
- general unrestricted languages
- context-free languages
- none of the other options
- regular languages

Consider the ***language*** L_3 generated by the following ***grammar*** (nonterminals ***S***, ***X***, ***Y***, axiom ***S***):

$S \rightarrow X \mid Y$

$X \rightarrow a\,Y\,b \mid a\,b$

$Y \rightarrow b\,X\,a \mid b\,a$

18. Say which is the simplest type of automaton that accepts L_3.

Select one:

- none of the other options
- nondeterministic push-down automaton
- deterministic push-down automaton
- finite-state automaton

19. This space is for writing your personal notes to answer other questions. These notes ***will not be corrected***.

20. Say which is the smallest family of languages to which L_3 belongs.

Select one:

- context-free languages
- none of the other alternatives
- general unrestricted languages
- regular languages

21. Write a formal and precise characterization of language L_3 . Do not use a grammar.

The following ***grammar*** defines a fragment of a ***programming language*** that includes ***expressions***. It supports:

- some common *arithmetic* operators: '+' for *addition*, '-' for *subtraction*, '*' for *multiplication*, '/' for *division*, '^' for *exponentiation*, unary '-' for *numeric negation*
- some *relational* operators: '==' for *equality*, '<' for *less-than*
- some *logical* operators: 'and', 'or', 'not' with the obvious meaning

<EXP> → 'false' | 'true' | <EXP> <BINOP> <EXP> | <UNOP> <EXP> | n

<BINOP> → '+' | '-' | '*' | '/' | '^' | '==' | '<' | 'and' | 'or'

<UNOP> → '-' | 'not'

where the terminal symbol ***n*** represents atomic expressions such as numeric literals and variables.

22. Is the above grammar ambiguous?

Select one:
- True
- False

23. Please provide a concise justification for your answer to the previous question.

24. Does the above grammar assign a relative priority to the various operators?

Select one:
- True
- False

25. According the the above grammar, the binary operators are

Select one:
- right associative
- none of the other options
- some right and some left associative
- left associative

Now imagine that, in the language to be defined by the grammar, the precedence among the operators is defined by the list below, starting from lower to higher priority

- or
- and
- < ==
- + - (binary)
- * /
- not - (unary)
- ^

Besides, exponentiation '^' is right-associative, while all the other binary operators are left-associative. Additionally, one can use parentheses '**(**' and '**)**' in the usual way to change the operator precedence.

Please modify the above grammar so to obtain a non-extended BNF grammar, non ambiguous, that allows for the use of parentheses as described above and assigns to the operators the relative precedence and the left- or right- associativity as specified above.

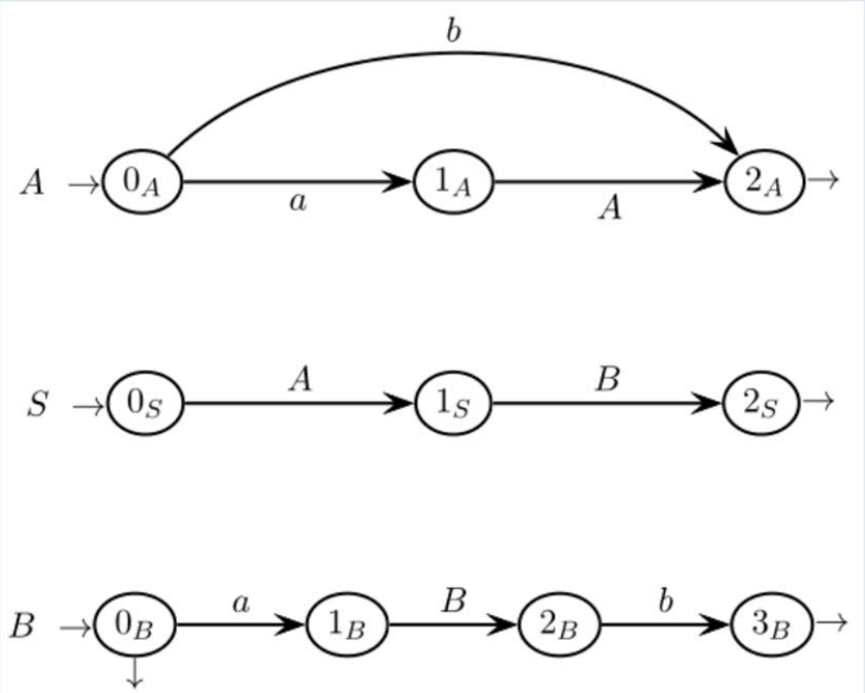
26.

(ADDITIONAL SPACE FOR PREVIOUS QUESTIONS)

PART 3 - Syntax analysis and parsing methodologies

27.

Consider the ***grammar G*** defined by the ***machine net*** below, with terminal alphabet ***a***, ***b*** and nonterminal alphabet ***A***, ***B***, ***S*** (axiom ***S***):

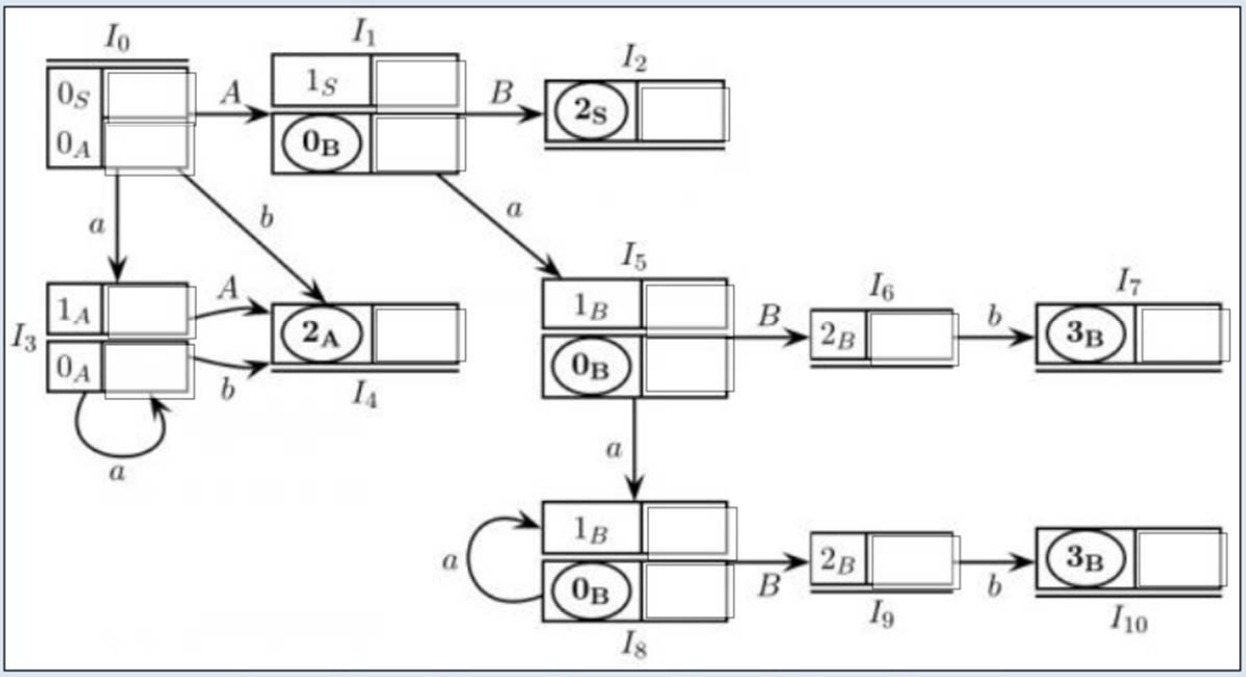


Select ***each statement*** below, related to the grammar ***G*** above and to its language ***L(G)***, that is ***true***.

Select one or more:

- ☐ Is grammar ***G recursive*** ?
- ☐ Is grammar ***G unmbiguous*** ?
- ☐ Is language ***L(G) finite*** ?
- ☐ Is grammar ***G ambiguous*** ?
- ☐ Is grammar ***G left-recursive*** ?
- ☐ Is language ***L(G) infinite*** ?

28. Please fill in the ***look-ahead sets*** in the items of the macro-states (m-states) of the following ***pilot*** automaton of the grammar ***G*** above:



a | b | t | a, b | a, t | b, t | a, b, t

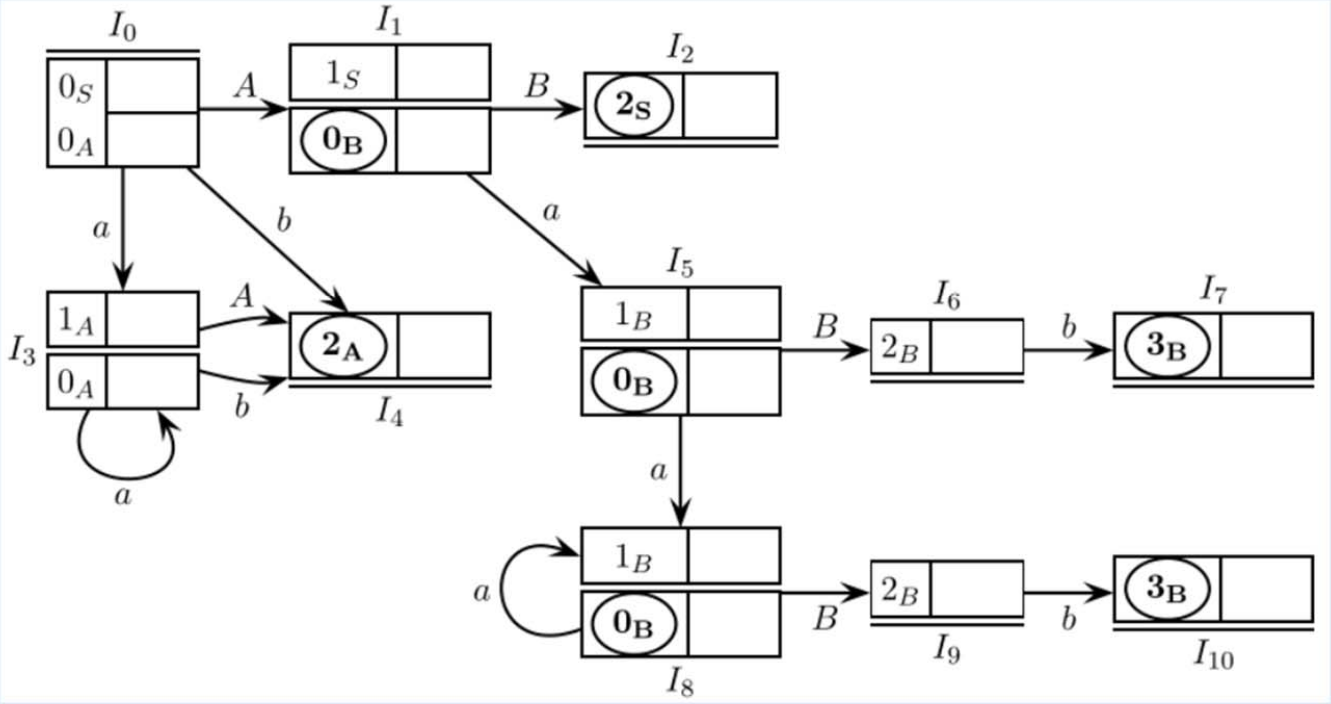
29. This space is for writing your personal notes to answer the previous questions. These notes ***will not be corrected***.

30. Say if the grammar ***G*** above is of ***type ELR(1)***.

Select one:
- True
- False

31.

For the ***pilot*** (reported below without look-ahead sets) of the grammar ***G*** above, the ***valid string abab*** is accepted deterministically. Simulate the execution of the ***ELR syntax analyzer*** of ***G*** and list the ***stack contents*** at eatch move (terminal shift, nonterminal shift or reduction). Use the ***template*** table below (first row already filled in).



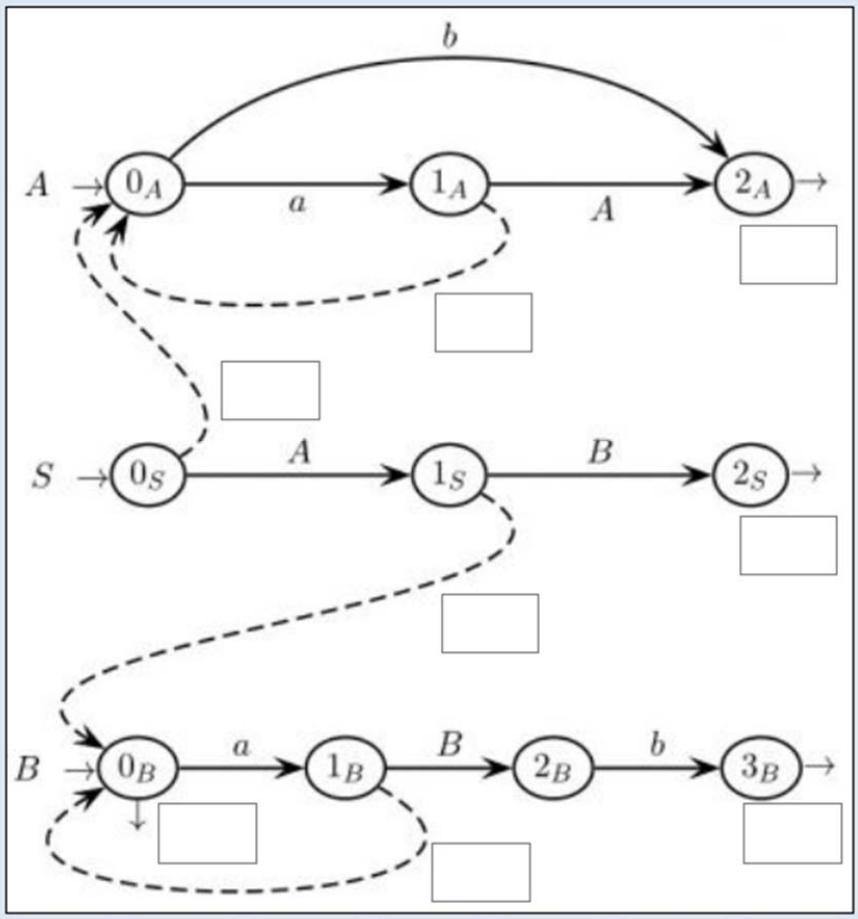
Fill the table below. Number of rows not significant:

row #	stack contents	input string left to analyze	description of the analyzer move
1	0	a b a b t	initial configuration
2	0 a 3	b a b t	terminal shift I0 → I3 on a
3			
4			
5			
6			
7			
8			
9			
10			
11			
12			
13			
14			
15			
16			

This space is for writing your personal notes to answer the previous questions. These notes ***will not be corrected***.

32.

33. Please fill in the guide ***sets on the call arcs and exit arrows*** of the mechine of the grammar ***G*** above.



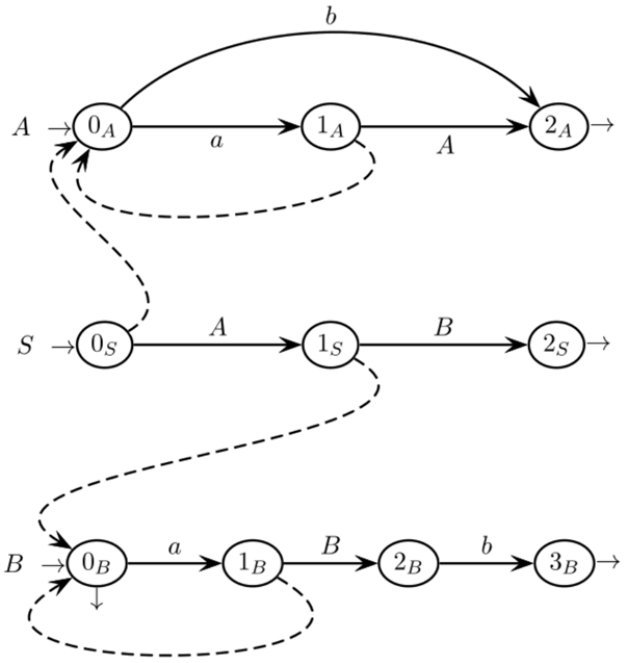
a	b	t	a, b	a, t	b, t	a, b, t

34. Say if the grammar ***G*** above is of ***type ELL(1)***.

Select one:

- True
- False

35. For the ***machine net with call arcs*** (reported below without guide sets) of the grammar ***G*** above, the ***valid string abab*** is accepted deterministically. Simulate the execution of the ***ELL syntax analyzer*** of ***G*** and list the ***stack contents*** at each move (shift, call or return). Use the ***template*** table below (first row already filled in).



Fill in the simulation table below. Number of rows not significant:

row #	stack contents ----------	input string left to analyze	description of the analyzer move
1	0S	a b a b t	initial configuration
2	...	...	...
3	...	...	...
4	...	...	...
5	...	...	...
6	...	....	...
7	...	...	...
8	...	...	...
9	...	...	...
10	...	...	...
11	...	...	...
12	...	...	...
13	...	...	...
14	...	...	...
15	...	...	...
16	...	...	...

36. This space is for writing your personal notes to answer the previous questions. These notes ***will not be corrected***.

37. Say if the language ***L(G)*** generated by the grammar ***G*** above is ***regular*** or not, and shortly ***justify*** your answer.

PART 4 - Language translation and semantic analysis

The following **source *grammar* G_S** (axiom ***S***):

S → ***A b S***

S → ***A b***

A → ***a A a***

A → ***a a***

generates strings of type $a^{2n_1} b\, a^{2n_2} b \ldots a^{2n_k} b$, with $k > 0$ and $n_i > 0$.

38. With reference to the ***source language*** defined by the grammar G_S above, a ***translation*** τ is defined as follows:

$$\tau\left(a^{2n_1}\, b \ldots a^{2n_k}\, b\right) = d\, c^{n_1} \ldots d\, c^{n_k}$$

For instance $\tau\left(a\,a\,a\,a\,b\,a\,a\,b\right) = d\,c\,c\,d\,c$.

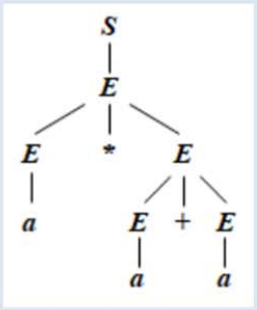
Write a ***target grammar*** G_T that, combined with G_S, provides a ***syntactic translation scheme*** that defines translation τ.

39. Write a ***regular translation expression*** R_T that defines the above translation τ.

The following grammar $\mathbf{G_A}$, with the nonterminal symbols ***S*** (axiom) and ***E***, defines a simple language for ***arithmetic expressions*** including binary addition '+' and multiplication '*', and constants represented by the terminal symbol ***a***.

1. $S \rightarrow E$
2. $E \rightarrow E * E$
3. $E \rightarrow E + E$
4. $E \rightarrow a$

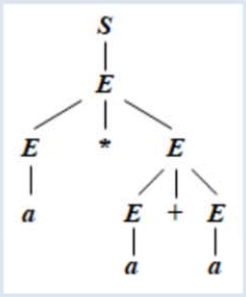
For instance, the string ***a * a + a*** admits the following syntax tree:



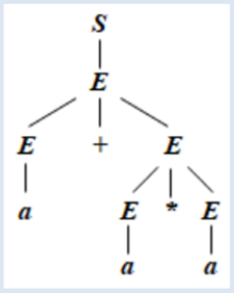
40.

With reference to the above grammar $\mathbf{G_A}$, we intend to provide a ***modified textual representation*** of the expressions it generates where, ***if*** any subexpression having '+' as the top operator is the operand (either left or right) of an expression or subexpression having '*' as the top operator, ***then*** the substring for the subexpression having '+' as the top operator must be surrounded by a ***pair of round parentheses*** '**(**' and '**)**'.

For instance, the expression having the following syntax tree:



is represented by the string ***a * (a + a)***, while the expression having the following syntax tree:



is represented by the string ***a + a * a***.

To this end, we introduce the following two attributes:

- ***p*** is a boolean ***right*** attribute associated with the nonterminal ***E***: it is true if and only if the immediately enclosing expression has '*' as the top operator
- ***s*** is a *left* attribute, of type string, associated with the nonterminals ***S*** and ***E***: for each nonterminal, the value of ***s*** is the string representation of the associated expression

Please ***write***, in the template provided below, the ***equations*** (semantic functions) for the attributes associated with every rule of the attribute grammar, in such a way that the value of the attribute ***s*** in the root ***S*** of the syntax tree is the desired string representation of the expression. Please denote the concatentation operator between strings by two consecutive dots '..'.

1. S_0 -> E_1 .....

2. E_0 -> E_1 * E_2 .....

3. E_0 -> E_1 + E_2 .....

4. E_0 -> a .....